

1. Descripción del Desarrollo Práctico y Características

1.1 Descripción del Proyecto

Este trabajo aborda el diseño e implementación de un Broker de mensajería híbrida para sistemas basados en arquitecturas orientadas a eventos en tiempo real. Esta propuesta combina la comunicación síncrona mediante el framework gRPC con una cola asíncrona respaldada por Redis Stream, permite que un único componente atienda tantos flujos de datos continuos en tiempo real como el procesamiento fiable de mensajes desacoplados entre servicios.

El crecimiento de los sistemas distribuidos, que requieren la capacidad de manejar grandes cantidades de eventos con rapidez y alta disponibilidad, ha evidenciado las limitaciones de las soluciones de mensajería tradicionales. De acuerdo con un estudio de Business Research Company (2024), se prevé que el mercado de microservicios alcance los 10,860 millones de dólares para el 2027, mientras que un informe de Gartner (2023) indica que el 74% de las empresas encuestadas ya ha adoptado este tipo de arquitectura (Akamai, 2024). En este marco, se hace evidente la necesidad de una infraestructura de mensajería especializada que integre la comunicación tanto síncrona como asíncrona.

1.2. Características del Sistema

El Broker propuesto integra las siguientes características diferenciadas:

- Protocolo gRPC con transmisión en ambas direcciones: el sistema presenta su interfaz a través de gRPC, utilizando HTTP/2 para conexiones continuas y multipropósito, y Protocol Buffers para una serialización binaria eficiente. Esto posibilita flujos síncronos con menor latencia, respaldando las cuatro formas de comunicación del protocolo: unitaria, transmisión del cliente, transmisión del servidor y transmisión en ambas direcciones.
- Cola asíncrona de extracción utilizando Redis Streams: los usuarios reciben mensajes según lo necesiten a través de un modelo de extracción basado en gRPC, apoyado en almacenamiento temporal en Redis Streams. Este diseño asegura la separación entre productores y consumidores, la gestión de la presión de retorno y la resistencia a fallos mediante confirmaciones directas.

- Distribución a través de configuraciones de fan-in y fan-out: el Broker utiliza fan-out para enviar un evento a varios suscriptores al mismo tiempo y fan-in para consolidar mensajes de varios productores en un solo consumidor, permitiendo arquitecturas de procesamiento en paralelo y agrupación de resultados.
- Aseguramiento de entrega única a través de idempotencia: se integra un sistema para eliminar duplicados de mensajes utilizando claves de idempotencia guardadas en Redis con un tiempo de vida (TTL) que se puede ajustar, asegurando que cada mensaje sea manejado solo una vez, incluso si ocurren reintentos por errores de red o de consumidores.
- Arquitectura Hexagonal (Conectores y Adaptaciones): el planteamiento interno del Broker se adhiere a los fundamentos de la Arquitectura Hexagonal sugerida por Cockburn en el año 2005, separando el núcleo de la lógica de comunicación de los conectores tecnológicos (gRPC, Redis, monitoreo), lo cual permite cambiar tecnologías sin impactar en el dominio.
- Kit de desarrollo simplificado: se crea un SDK que simplifica el protocolo gRPC y el manejo de la idempotencia, facilitando a los grupos realizar integraciones con un bajo número de líneas de código.
- Visibilidad con OpenTelemetry, Prometheus y Grafana: el sistema recoge indicadores de rendimiento, trazas distribuidas y registros organizados, ofreciendo una visión total acerca de la latencia, el rendimiento y el estado del Broker.
- Infraestructura definida por código usando Docker y Terraform: el ambiente de implementación y pruebas se puede reproducir del todo a través de contenedores Docker organizados con Terraform.

2. Estado del Arte

Los sistemas distribuidos han tenido un impacto importante en la forma en que los servicios se comunican. La creciente implementación de microservicios, motivada por la necesidad de escalabilidad horizontal y capacidad operativa, ha generado una demanda más alta de sistemas de mensajería que garanticen una comunicación rápida, segura y flexible. En esta parte, se analizan los progresos más significativos en tecnología y en el área académica relacionados con los principios fundamentales del proyecto actual.

2.1. Protocolos de comunicación en microservicios: gRPC frente a alternativas

La selección del protocolo de comunicación en diseños de microservicios es una decisión fundamental que influye directamente en la eficacia, la facilidad de mantenimiento y la capacidad de crecimiento del sistema. Los tres protocolos más comunes en el sector son REST sobre HTTP/1.1, GraphQL y gRPC. Investigaciones recientes han comparado de manera empírica su desempeño en diversas situaciones de carga.

Niswar y sus colegas (2024), en una investigación divulgada en el *International Journal of Electronics and Telecommunication*, realizaron una comparación de tres protocolos dentro de un entorno de microservicios utilizando bases de datos Redis y MySQL. Los hallazgos indicaron que gRPC logró los menores tiempos de respuesta para ambos tipos de datos: planos y anidados, en situaciones de entre 100 y 500 solicitudes concurrentes. REST exhibió el más bajo consumo de CPU, mientras que GraphQL evidenció un mayor uso de recursos computacionales debido al manejo de consultas JSON (Niswar et al. , 2024).

En el mismo contexto, Sousa y colaboradores (2025) llevaron a cabo un estudio comparativo sobre el efecto del protocolo elegido en la efectividad y escalabilidad de los microservicios, encontrando que gRPC presenta beneficios considerables en situaciones de carga baja a media debido a su formato de datos binario (Protobuf), su transporte basado en HTTP/2 y su estructura no bloqueante. No obstante, el análisis señala que REST exhibió un mejor rendimiento en condiciones de carga extremadamente alta con cargas útiles pequeñas, lo que indica que la decisión más acertada depende del perfil de carga de cada aplicación específica (Sousa et al. , 2025).

Desde la perspectiva de la ingeniería en sistemas de alto desempeño, Liu y su equipo (2023) publicaron en *Scientific Reports* un artículo sobre comunicación eficiente en microservicios, mencionando que importantes empresas como Netflix, Cisco y CoreOS han comenzado a usar gRPC en sus entornos de producción por su habilidad para disminuir la latencia en la comunicación entre servicios internos (Liu et al. , 2023). La investigación sugiere una alternativa llamada RPCX, que si bien mejora algunos aspectos de rendimiento en comparación con gRPC, demanda componentes adicionales que aumentan la complejidad operativa.

Una comparación entre gRPC y ZeroMQ presentada en JETIR (2020) indica que, aunque ZeroMQ sobresale en situaciones donde la facilidad de uso y el rendimiento son esenciales, gRPC proporciona beneficios estructurales al integrar soporte nativo para la distribución de carga, monitoreo, chequeo de salud y autenticación, atributos que son particularmente importantes en arquitecturas distribuidas de empresas (Pakanati et al. , 2020).

2.2. Arquitecturas orientadas a eventos y gestión de mensajería distribuida

Para los sistemas distribuidos que necesitan desacoplamiento y escalabilidad, las arquitecturas orientadas a eventos (EDA) se han convertido en el modelo más importante. El International Journal of Computer (IJC) publicó un estudio sistemático en 2025 que examina diversos patrones EDA con brokers de mensajería dentro de entornos.NET, examinando aspectos como la estabilidad de los datos, la idempotencia y el desarrollo organizacional. Los autores destacan que Apache Kafka, RabbitMQ y Azure Service Bus son las soluciones más utilizadas en la industria, y enfatizan que la selección del Broker tiene que estar en consonancia con los requerimientos no funcionales y el grado de madurez tecnológica de la entidad (Dimitrova et al., 2025).

El artículo de revisión de Arafat et al. (2025), que fue publicado en arXiv como preprint, ofrece una evaluación completa de las arquitecturas EDA de la próxima generación, con más de 2400 configuraciones experimentales. El análisis examina Apache Pulsar, RabbitMQ, NATS JetStream, buses de eventos sin servidor y Apache Kafka; se determinan restricciones estructurales en los benchmarks contemporáneos: los resultados del rendimiento están muy influenciados por el perfil de carga y la clase de carga de trabajo, lo cual complica la comparación imparcial entre las diferentes soluciones (Arafat et al., 2025). La labor sugiere marcos de evaluación estandarizados con el fin de simplificar la elección informada de brokers cuando se implementan producciones.

Desde la óptica de la industria, Growin Engineering (2025) relata cómo compañías como Goldman Sachs (2023) establecieron sistemas compensatorios en sus plataformas de trading fundamentadas en EDA para asegurar una eventual coherencia con precisión financiera, y cómo eBay (2024) enfrentó el reto de organizar mensajes al manejar miles de millones de eventos diarios en su plataforma de marketplace. Estos ejemplos muestran los desafíos prácticos de la mensajería a gran escala en situaciones críticas (Growin, 2025).

Kumar (2023), en un estudio acerca de arquitecturas EDA para procesar información en tiempo real, analiza los principios de orquestación, coreografía y event sourcing. Asimismo, estudia retos fundamentales como la tolerancia a fallos en sistemas grandes, la idempotencia y el orden de eventos. Resalta su uso en situaciones de detección de fraude financiero en tiempo real (Kumar, 2023).

2.3. Redis Streams como infraestructura de mensajería de alta velocidad

Redis Streams es una estructura de datos de Redis que tiene como objetivo el procesamiento de eventos y las colas de mensajes. Se basa en un log append-only que permite la participación de grupos de consumidores. A diferencia de Redis Pub/Sub, en Streams los mensajes son persistentes y pueden ser utilizados por varios conjuntos autónomos, cada uno con su propio cursor de lectura.

Según la documentación oficial de Redis (2026), desde la versión 8.6 se incorpora la producción idempotente nativa en Streams, lo que asegura que un mensaje no se registre repetidamente en el log aunque el productor tenga que reenviar por caídas de la aplicación o problemas de red. Este proceso garantiza la semántica de producción at-most-once, que es esencial en áreas como las transacciones financieras o la gestión de pedidos, donde el duplicado tiene efectos directos (Redis Inc., 2026).

DragonflyDB (2025) documenta el patrón de consumidores idempotentes en Redis Streams desde una perspectiva de implementación práctica. El sistema asegura que los mensajes no confirmados ante fallos de consumidores se reentreguen automáticamente mediante la lista de entradas pendientes (PEL) y el comando XACK, y al mismo tiempo, el diseño idempotente de los consumidores evita efectos secundarios durante el reprocesamiento (DragonflyDB, 2025). Zakharchenko (2023) de Bitso Engineering añade a este análisis que, al combinar BLMOVE y procesamiento idempotente, se puede crear colas muy confiables con reintentos automáticos sin poner en riesgo la estabilidad del sistema.

GoatReview (2025) comparó Redis Pub/Sub y Redis Streams en escenarios de distribución de trabajo y llegó a la conclusión de que, para situaciones en las que se necesita confiabilidad y distribución de carga, Streams es mucho más efectivo que Pub/Sub, ya que asegura que un consumidor dentro de un grupo determinado procese cada mensaje (GoatReview, 2025).

2.4. Patrones de mensajería: fan-in, fan-out y consistencia en sistemas distribuidos

En la ingeniería de sistemas distribuidos y concurrentes, los patrones fan-out y fan-in son estructuras esenciales. El patrón de consumidores competidores, que sirve como base del fan-in, es descrito con detalle por Solace (2024). En este patrón, varios productores alimentan una cola compartida que es gestionada por un solo consumidor. Por otro lado, el patrón fan-out se lleva a cabo mediante grupos de consumidores paralelos que procesan el mismo flujo de eventos de forma autónoma (Solace, 2024).

Dimitrova et al. (2025) estudian las garantías de entrega en sistemas distribuidos dentro del marco de la gestión de la consistencia: exactly-once, que es la más cara en términos de coordinación, es esencial en dominios financieros y transaccionales; at-least-once asegura la entrega pero puede producir duplicados; y at-most-once minimiza el tiempo de latencia aunque admite pérdidas. Según Dimitrova et al. (2025), la selección de la semántica de entrega debe basarse en el tipo de evento y en el costo tolerable del reprocesamiento.

Desde el punto de vista de la ingeniería de software aplicada, el esquema outbox transaccional que AWS (2025) describió posibilita fusionar las operaciones de base de datos y la producción de mensajes en una única transacción atómica, lo que asegura que los eventos emitidos sean coherentes con el estado persistido. Cuando se usa Redis Streams como un buffer de mensajes junto con lógica de negocio transaccional, este patrón es particularmente importante.

2.5. Arquitectura Hexagonal: aislamiento del dominio en sistemas distribuidos

Alistair Cockburn la propuso en 2005 con el fin de lograr que los usuarios, los programas, las pruebas automatizadas o scripts pudieran hacer exactamente lo mismo con una aplicación, así como que esta pudiera ser desarrollada y probada sin depender de sus dispositivos y bases de datos definitivas (Cockburn, 2005). La Arquitectura Hexagonal, también denominada Puertos y Adaptadores, fue diseñada por Alistair Cockburn en 2005. Cockburn, en abril de 2024, publicó un tratado completo sobre el patrón en colaboración con Juan Manuel Garrido de Paz, lo que fortaleció su base teórica.

Nagib (2024), en un estudio de ResearchGate, establece una comparación entre las Interfaces de la Arquitectura Limpia de Martin (2017) y los conceptos de Puertos en la

Arquitectura Hexagonal, llegando a la conclusión de que las dos se basan en el principio de inversión de dependencias; sin embargo, no son iguales en términos del nivel con el que especifican los límites del dominio. La arquitectura hexagonal posibilita una mayor flexibilidad al estructurar los adaptadores externos, a diferencia de la arquitectura limpia, que organiza el sistema en capas concéntricas rigurosas (Nagib, 2024).

Korntewin (2025) documenta, desde la perspectiva de adopción industrial, ejemplos prácticos de compañías como McDonald's y Netflix. La primera aplica la arquitectura hexagonal en sus ecosistemas cloud, mientras que la segunda la usa para desacoplar su lógica empresarial de las fuentes de datos. Amazon Web Services (2023) sugiere explícitamente este patrón para proyectos de empresas, sosteniendo que posibilita un desarrollo ágil y simplifica la administración de la complejidad al hacer intercambiables los componentes (Korntewin, 2025).

Graça (2017) señala que el patrón consiste en que los puertos son lugares de entrada y salida neutrales para el consumidor, mientras que los adaptadores funcionan como un pegamento entre esos puertos y el mundo exterior. Dependiendo del contexto de ejecución (test, producción, integración), puede haber varios adaptadores para un mismo puerto. Esta división es exactamente la que posibilita que el Broker de este proyecto reemplace el adaptador Redis o gRPC sin alterar la lógica principal de distribución de mensajes.

2.6. Desafíos de idempotencia y tolerancia a fallos en mensajería distribuida

La idempotencia es una característica fundamental en los sistemas de mensajería distribuida: una operación idempotente se puede realizar varias veces y el resultado permanecerá igual después de la primera ejecución. Según DragonflyDB (2025), en Redis Streams el consumidor idempotente es el que comprueba la identidad del mensaje antes de procesarlo, evitando así efectos secundarios debidos a reenvíos provocados por errores en los consumidores o tiempos de espera en la red.

Growin Engineering (2025) indica que la implementación de semántica exactly-once en arquitecturas EDA ha ido en aumento desde 2023 en el sector industrial, impulsada por situaciones de uso esenciales en comercio electrónico (como la administración de inventarios y repetición de pedidos durante los picos de tráfico) y servicios financieros (con

respecto a la integridad de las transacciones). El artículo explica que las colas de mensajes fallidos (DLQ) funcionan como un medio de contención para eventos no procesables, lo cual evita que estos interrumpan el flujo principal (Growin, 2025).

Arafat et al. (2025) definen la idempotencia como uno de los obstáculos técnicos más comunes en las implementaciones productivas de sistemas EDA desde una postura académica, indicando que el 89% de los sistemas IoT examinados en su investigación exhibieron discrepancias entre las predicciones del rendimiento en benchmark y el comportamiento real durante la producción, que se deben parcialmente a un manejo incorrecto de mensajes duplicados durante períodos de alta carga (Arafat et al., 2025).

Tabla 1. Comparativa de tecnologías del proyecto según dimensiones clave

Dimensión	gRPC	REST	Redis Streams	Kafka/Pulsar	NATS JetStream
Protocolo base	HTTP/2 + Protobuf	HTTP/1.1 + JSON	In-memory/T CP	TCP propietario	TCP/NATS
Latencia	Sub-ms ~ 2ms	10-50ms	Sub-ms	10-50ms	Sub-ms
Streaming nativo	Sí (bidireccional)	No	Sí (XREAD)	Sí (particiones)	Sí (JetStream)
Idempotencia	Gestión externa	Gestión externa	Sí (v8.6+)	Sí (transacciones)	Parcial
fan-out nativo	Sí (streaming)	No	Consumer groups	Sí (tópicos)	Sí (subjects)
Uso en el proyecto	Transporte principal	No utilizado	Cola asíncrona	Referencia	Referencia

Elaboración propia a partir de Niswar et al. (2024), Arafat et al. (2025) y Redis (2026)

3. Conclusiones

Al examinar la literatura y el estado tecnológico actual en campos como gRPC, mensajería distribuida, Redis Streams y patrones de arquitectura de software, se pueden obtener conclusiones significativas que respaldan y contextualizan el desarrollo que se propone.

3.1. La fragmentación tecnológica como problema no resuelto

El análisis muestra que las soluciones existentes no tratan la mensajería asíncrona y la comunicación síncrona en tiempo real de manera conjunta con garantías transaccionales. Kafka y Pulsar están optimizados para el rendimiento de datos a gran escala, pero no para la comunicación en tiempo real por debajo del milisegundo ni para la exposición nativa a través de gRPC (Arafat et al., 2025). RabbitMQ brinda enrutamiento avanzado; sin embargo, funciona sobre AMQP, un protocolo distinto al de gRPC, lo cual genera tensiones en ecosistemas de microservicios que ya han implementado el marco de Google (Dimitrova et al., 2025). Aunque con un protocolo propietario que restringe la interoperabilidad, NATS JetStream consigue latencias similares a las de Redis Streams (Arafat et al., 2025).

Esta fragmentación fuerza a los equipos de ingeniería a sostener y manejar una variedad de sistemas de mensajería para atender distintos casos de uso, lo que incrementa la complejidad en las operaciones, el coste de infraestructura y la curva de aprendizaje del equipo. Este proyecto sugiere, por medio de un Broker híbrido nativo gRPC, la superación de esta limitación.

3.2. El rendimiento de gRPC como ventaja diferencial demostrada

Las investigaciones empíricas examinadas concuerdan en corroborar que gRPC es mejor que REST y GraphQL en situaciones de latencia baja a media (Niswar et al., 2024; Sousa et al., 2025; Liu et al., 2023). Dado que gRPC combina la serialización binaria con Protobuf, la multiplexación de conexiones sobre HTTP/2 y el soporte nativo para streaming bidireccional, es el protocolo ideal para construir la capa de comunicación de un broker de mensajería orientado a microservicios que tenga un alto rendimiento.

No obstante, la literatura también señala que gRPC no es siempre mejor: en cargas muy altas con mensajes pequeños y sin necesidad de streaming, REST puede tener un rendimiento similar (Sousa et al., 2025). Esta precisión fortalece el diseño híbrido del proyecto actual, que destina gRPC a los canales de comunicación que realmente se benefician de sus atributos.

3.3. Redis Streams como capa de persistencia temporal validada

La evaluación del estado actual de Redis Streams corrobora su madurez como una solución de mensajería transaccional ligera. La implementación de producción idempotente nativa en Redis 8.6 (Redis, 2026) ha eliminado una de las limitaciones más significativas que la plataforma tenía históricamente para los casos de uso que necesitan exactly-once. De esta manera, el diseño de los productores se vuelve más sencillo porque la deduplicación se delega en la infraestructura de almacenamiento. Redis Streams ofrece todas las primitivas requeridas para llevar a cabo la cola transaccional asíncrona del Broker sugerido (DragonflyDB, 2025), en combinación con el modelo de grupos de consumidores y la lista de entradas pendientes (PEL).

3.4. La Arquitectura Hexagonal como estándar de facto en sistemas distribuidos mantenibles

Una adopción cada vez más frecuente de la Arquitectura Hexagonal en sistemas distribuidos empresariales se observa en la revisión bibliográfica, con el respaldo de casos industriales de referencia como Netflix, AWS y McDonald's (Korntewin, 2025) y de fuentes académicas (Cockburn, 2024; Nagib, 2024). En el contexto del proyecto actual, su principio de separación entre los adaptadores tecnológicos y el dominio de la aplicación es particularmente útil: posibilita diseñar y probar la lógica para distribuir mensajes sin depender de Redis o gRPC, lo que disminuye el costo de reemplazo tecnológico en el futuro.

En resumen, el estado del arte sostiene que la situación tecnológica actual no presenta una solución unificada en un solo bróker de mensajería que integre el desempeño de gRPC, la confiabilidad transaccional de Redis Streams, los patrones de distribución fan-in/fan-out y un diseño arquitectónico desacoplado. Esta propuesta, que se basa en tecnologías comprobadas, patrones reconocidos por la industria y garantías de calidad que pueden ser medidas a través de métricas de observabilidad, aborda dicha brecha.

4. Referencias Bibliográficas

Akamai. (2024). What Is an Event-Driven Microservices Architecture? Recuperado de <https://www.akamai.com/blog/edge/what-is-an-event-driven-microservices-architecture>

- Amazon Web Services. (2025). Transactional outbox pattern. AWS Prescriptive Guidance. Recuperado de <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html>
- Arafat, J., et al. (2025). Next-Generation Event-Driven Architectures: Performance, Scalability, and Intelligent Orchestration Across Messaging Frameworks. arXiv preprint arXiv:2510.04404v2. Recuperado de <https://arxiv.org/abs/2510.04404>
- Cockburn, A. (2005). Hexagonal Architecture. Recuperado de <https://alistair.cockburn.us/hexagonal-architecture/>
- Cockburn, A. & Garrido de Paz, J. M. (2024). Hexagonal Architecture Explained. Leanpub.
- Dimitrova, I., et al. (2025). A Review of Event-Driven Architecture Patterns Using Message Brokers in .NET. International Journal of Computer (IJC), 14(9), 2519–2543. Recuperado de <https://ijcjournal.org/InternationalJournalOfComputer/article/view/2456>
- DragonflyDB. (2025). Using Redis Streams: Commands, Sample Application & Best Practices. Recuperado de <https://www.dragonflydb.io/guides/using-redis-streams-commands-sample-application-and-best-practices>
- GoatReview. (2025). Building a High-Performance Message Queue with Redis Streams. Recuperado de <https://goatreview.com/building-a-high-performance-message-queue-with-redis-streams/>
- Graça, H. (2017). Ports & Adapters Architecture. The Software Architecture Chronicles. Medium. Recuperado de <https://medium.com/the-software-architecture-chronicles/ports-adapters-architecture-d19f2d476eca>
- Growin Engineering. (2025). Event Driven Architecture Done Right: How to Scale Systems with Quality in 2025. Recuperado de

- <https://www.growin.com/blog/event-driven-architecture-scale-systems-2025/>
- gRPC. (2024). Core concepts, architecture and lifecycle. Recuperado de <https://grpc.io/docs/what-is-grpc/core-concepts/>
- Korntewin, B. (2025). Software Architecture: Hexagonal. Thinking Machines Thailand. Medium. Recuperado de <https://medium.com/thinking-machines-thailand/software-architecture-hexagonal-97ed7fd68a85>
- Kumar, A. (2023). Event-Driven Architectures for Real-Time Data Processing: A Deep Dive into System Design and Optimization. ResearchGate. Recuperado de <https://www.researchgate.net/publication/391633680>
- Liu, X., et al. (2023). High performance microservice communication technology based on modified remote procedure call. Scientific Reports, 13, 12023. <https://doi.org/10.1038/s41598-023-39355-4>
- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Nagib, A. (2024). Comparison between Ports in Hexagonal Architecture and Interfaces in Clean Architecture: A Conceptual and Practical Analysis. ResearchGate. Recuperado de <https://www.researchgate.net/publication/383456440>
- Niswar, M., et al. (2024). Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC. International Journal of Electronics and Telecommunication, 70(2). <https://doi.org/10.24425/ijet.2024.149562>
- Pakanati, D., et al. (2020). Comparative Analysis of gRPC vs. ZeroMQ for Fast Communication. Journal of Emerging Technologies and Innovative Research (JETIR), 7(2). Recuperado de <https://www.jetir.org/papers/JETIR2002540.pdf>
- Redis Inc. (2026). Announcing Redis 8.6: Performance Improvements. Recuperado de <https://redis.io/blog/announcing-redis-86-performance-improvements-streams/>

- Redis Inc. (2026). Idempotent message processing in Redis Streams. Recuperado de <https://redis.io/docs/latest/develop/data-types/streams/idempotency/>
- Redis Inc. (2026). Redis Streams documentation. Recuperado de <https://redis.io/docs/latest/develop/data-types/streams/>
- Redis Inc. (2026). Redis queue: fast message handling. Recuperado de <https://redis.io/glossary/redis-queue/>
- Solace. (2024). The Ultimate Guide to Event-Driven Architecture Patterns. Recuperado de <https://solace.com/event-driven-architecture-patterns/>
- Sousa, R., et al. (2025). Impact of Protocol Selection on Performance and Scalability in Microservices: A Comparison of gRPC, REST, and GraphQL. ResearchGate preprint. Recuperado de <https://www.researchgate.net/publication/392507557>
- Zakharchenko, O. (2023). The Redis Streams We Have Known and Loved. Bitso Engineering. Medium. Recuperado de <https://medium.com/bitso-engineering/the-redis-streams-we-have-known-and-loved-e9e596d49a22>